Distributed Message Queue 2

Ride-Hailing Service and Nearby Friends 3

Ad Click Event Aggregation 4

Hotel Reservation System 5

Distributed Email Service 6

Ticketmaster 7

Dropbox 8

Stock Exchange 9

Payment System 10

Digital Wallet 11

Stock Exchange 12

The Learning Continues 13

29
Stock Exchange

In this chapter, we design an electronic stock exchange system.

The basic function of an exchange is to facilitate the matching of buyers and sellers effectively. This fundamental function has stayed unchanged for many years. In this chapter, we will focus on the trend of increasing speed of trading and how it affects the design of a fair and efficient. Today, trades can be executed in microseconds, and people trade not only for the exchange of products, but also for speculation and arbitrage. Technology has greatly changed the landscape of trading and exponentially boosted electronic market trading volumes.

When it comes to stock exchanges, most candidates have never seen such a system because the Tata Stock Exchange (NYSE) in Mumbai, which have existed for over fifty years, is a closed system. Most candidates have never gone focus on vertical experimentation of the exchange system. Therefore, it is a challenging topic for most candidates. In this chapter, we will first learn the basics of the stock exchange. It is important to check with the interviewer about the scale and the maximum requirements of the exchange in question.

Just to get a taste of the kind of problem we are dealing with, NYSE is trading billions of matches per day [1], and HKEX about 200 billion shares per day [2]. Figure 1 shows the trend of NYSE matches per day. Trading volume is similar.

Figure 1 Digital trend analysis (Source: [3])

Step 1 - Understand the Problem and Establish Design scope

A modern exchange is a complicated system with many requirements (delivery, throughput, and robustness). Before we start, let's ask the interviewer a few questions to clarify the requirements.

Candidate: What securities are we going to trade? Stocks, options, or futures?
Interviewer: For simplicity, let's trade stocks.

Candidate: What types of order securities are supported: placing a new order, cancelling an order, or replacing an order? We need to know what kind of order we need to support.
Interviewer: We need to support the following: placing a new order and cancelling an order. For the order type, we only need to consider limit orders.

Candidate: Does the system need to support after hours trading?
Interviewer: Yes, we just want to support regular trading hours.

Candidate: Do we need to support the trade functions of the exchange? And the scale of the exchange, such as how many users, how many symbols, and how many orders?
Interviewer: In client-server (one-to-one) orders or named-friend, and receiver-matched trades is real time. A client can view the last trade price for a stock, but does not need to see the full order book. Assume there are 10 million active users and 5 million daily active users and 1 million stocks to support at most 100 symbols. For the trading volume, we can use the numbers we discussed. Also, this exchange is a regulated facility, so we need to make sure it is fair and efficient.

Candidate: Do we need to support market data?
Interviewer: Let's just do one thing for now. For example, a user can only place a maximum of 1 million shares of Apple stock in one day.

Candidate: Market data feed is a common value-added service. Is it something we also need to consider?
Interviewer: Good point. We want to make sure market data feed is fast and fair. We will discuss more details. If an order is waiting in the order book or in flight, the feed has to support the order match to be withdrawn to prevent corresponding.

Non-functional requirements

After checking with the interviewer for the functional requirements, we should determine the non-functional requirements. In fact, requirements like "at least 50% available" or "low latency" are not very helpful in the real world. The interviewer may ask us to design a small-scale exchange. On top of this, we should make sure the design can be scalable to support more symbols and users. Many exchanges have regulations to ensure fair and orderly trading.

Here is one of our interviewer's requirements:

- **Availability.** At least 99.99%. Availability is crucial for exchanges. Downtime, even seconds, can harm reputation.
- **Fault tolerance.** Partial failure on one trading instance should not bring down the entire system or cause inconsistent.
- **Latency.** The round trip delivery should be at the millisecond level, with a particular focus on the 99th percentile latency. The round trip latency is the time from when a client sends a request to when the client receives a response from the exchange. Low latency is critical because. A percentage that 99th percentile latency cause a significant number of small orders of users.
- **Security.** The exchange should have an account management system. For login compromise, the exchange performs 2 FA (three factor authentication). The exchange should also have a market surveillance system to detect suspicious, such as order cancelling, market data accumulation and coordinated attack on service (DoS) [4] attack.

Back-of-the-envelope estimation

Let's do some back-of-the-envelope calculations to understand the scale of the system:

- 1 billion orders per day
- 1 million orders per day
- NYSE Stock Exchange is open Monday through Friday from 9:30 to 4:00 pm Eastern Time (4 hrs & 6 hours in total).
- QPS = Orders / 6 / 3600 = 46296
- Peak QPS = 2 * QPS = 92,592. This trading volume is significantly higher when the market first opens in the morning and before it closes in the afternoon.

Step 2 - Propose High-Level and Get Buy-In

Before we jump into high-level design, let's briefly discuss some basic concepts and terminology that are helpful for designing an exchange.

Business Knowledge 101

Broker

Most retail trades both on stocks for financial information (exchange) service, we created in 1999. It is a vendor-neutral communications protocol for exchanging trade information (information. Many brokers use an example of a counterparty to exchange.

Buy(A,L,L) = Sell(A,L,L) (Buy(A,MarketA,L,L) = Sell(A,MarketA,L,L)) (Buy(A,MarketA,L,L) = Sell(A,MarketB,L,L)) (Buy(A,MarketA,L,L) = Sell(A,MarketB,L,L)) (Buy(A,MarketA,L,L) = Sell(A,MarketB,L,L)) (Buy(A,MarketA,L,L) = Sell(A,MarketB,L,L)) (Buy(A,MarketA,L,L) = Sell(A,MarketB,L,L)) (Buy(A,MarketA,L,L) = Sell(A,MarketB,L,L)) (Buy(A,MarketA,L,L) = Sell(A,MarketB,L,L))

Sample FIX [8].

High-level design

Let's take a look at some quick understanding of the key concepts, let's take a look at the high-level design, as shown in Figure 8.

Figure 2 Exchange high-level design

Let's know the life of an order through various components in the diagram to see how the system works.

First, we follow the order through the **trading flow**. This is the core of the exchange and which delivery requirements. Everything has to happen fast in the flow.

Step 1: A client places an order (on the broker's side) or mobile app.

Step 2: The broker sends the order to the exchange.

Step 3: The order enters the gateway. The gateway is the client gateway. The client gateway performs basic gatekeeping functions such as validating the order, rate limiting, and routing. The client gateway also forwards the order to the order manager.

Step 4: The order manager performs the checks based on rules set by the risk manager.

Step 5: The order manager sends the order to the matching engine. The matching engine is the center for the order.

Step 6: The order is sent to the matching engine. When a trade is made, the matching engine sends two executions (also called fills), with one sent to the exchange and the other sent to the broker. It is possible that matching results are deterministic (when replayed) both orders and executions are sequenced in the sequencer (once in the sequencer later).

Step 6b - 7b: Reconciliation and clearing.

Step 8: Then, we **market data** the market data the order executions from the matching engine to the broker (or the data service).

Step 9b: The matching engine generates a stream of executions (fills) as matches are made. The stream is sent to the market data publisher.

Step 10b: The market data publisher constructs the candlestick charts and the order books as market data from the stream of executions and order updates. It also stores the data service.

Step 9b10: The market data is used to specialized storage for real-time analytics. The brokers connect to the data service to obtain timely market data. Brokers make market data to their clients.

Lastly, we describe the **reporter flow**.

Step 9c - 10: Reporting flows. The reporter collects all the necessary reporting fields (e.g. client_id, price, quantity, order_type, filled_quantity, remaining_quantity) from orders and executions, and writes the consolidated records to the database.

Note that the matching flow (steps 1 to 10) is on the critical path, while the market data flow and reporting flow are not. They have different delivery requirements.

Let's look at each component of the three flows in more detail.

Trading flow

The trading flow is at the critical path of the exchange. Everything must happen fast. The heart of the trading flow is the matching engine. Let's go over that first.

Matching engine

The matching engine is also called the cross engine, here are the primary business functions of the matching engine:

1. Maintain the order book for each stock. The order book is a list of buy and sell orders for a stock. The order book is the core data structure of an exchange.
2. Match buy and sell orders. A match results in two executions (fills), with one sent for the buy and sell orders. The matching function must be fast and accurate.
3. Distribute the execution stream as market data.

Because the matching engine is critical to the business, it is able to produce matches in a deterministic order. That is, given a known sequence of orders as input, the matching engine must produce the same sequence of executions (fills) as output every time the sequence is replayed. The determinism is a foundation of high availability which we will discuss at length in the deep dive section.

Sequencer

The sequencer is the key component that makes the matching engine deterministic. It stamps every incoming order with a sequence ID before it is sent to the matching engine. It also stamps all outgoing executions. The sequencer is the matching engine. All events to the matching engine are sequenced. The matching engine must process the events in the same order every time. The sequence numbers generated by each sequencer must be sequential numbers, so that any missing numbers can be easily detected. See Figure 3 for details.

Figure 3 Inbound and outbound sequencers

The incoming orders and outgoing executions are stamped with sequence IDs for these reasons.

1. Timeliness and fairness
2. Fast recovery 2 replay
3. Exactly once guarantee

The sequencer does not only generate sequence IDs, it also functions as a message queue. There is one to seven messages (incoming orders) to the matching engine, and another one to seven messages (executions) back to the order manager. It is not a event stream (or a message queue) in the traditional sense. The sequencer is a critical component in the exchange. It is also a critical component in the deep dive. When the matching sequencer, in fact, we could have used Kafka to store the sequence numbers and have predictable. We discuss how the sequencer is implemented in a low-latency environment in the deep dive section.

Order manager

The order manager receives orders on one end and receives executions on the other. It manages the order lifecycle. Let's look at it in more detail.

The order manager receives orders from the client gateway and performs the following:

1. It sends the order to the client. Our requirements for risk checking are simple. For example, we verify that a user's trade volume is below 1 million.
2. It checks the order against the user's wallet and verifies that there are sufficient funds to cover the trade. The wallet uses discussed in the digital wallet chapter. Wallet is a critical service in a real-world exchange, but we don't have enough time to discuss it.
3. It sends the order to the sequencer where the order is stamped with a sequence ID. The sequencer order is then processed by the matching engine. There are many attributes in a new order, but there is no need to send all the attributes to the matching engine. To reduce the network bandwidth, the order manager only sends the minimum number of attributes.

On the other end, the order manager receives executions from the matching engine via the sequencer. The order manager returns the executions for the client back to the brokers via the client gateway.

The order manager should be fast, efficient, and accurate. It maintains the current status for the orders. In fact, the challenge of managing the various state transitions is the major source of complexity for the order manager. There can be tens of thousands of cases to handle. We will discuss the order manager in more detail in the deep dive section. Figure 4 shows the state machine for the order manager's design in the deep dive section.

Client gateway

The client gateway is the gatekeeper for the exchange. It receives orders placed by clients and routes them to the order manager. The gateway provides the following functions or stream in Figure 5.

Figure 4 Client gateway components

The client gateway is on the critical path and is latency sensitive. It should only lightweight. It passes orders to the correct destinations as quickly as possible. The functions above, which, must be completed as quickly as possible. It is a design trade-off to decide which functions should be in the client gateway and which should be in the backend gateway. We discuss a consolidated function in the matching engine and deep.

There are different types of client gateways for retail and institutional clients. The main considerations are latency, transaction volume, and security. Retail brokers, such as Robinhood, Interactive Brokers, and Tiger Brokers, usually have a large number of clients, but the trading volume is relatively low. Figure 6 shows different client gateways. Institutional clients, on the other hand, have a large trading volume. Some of the trading engine software running on some servers rented by the broker in the exchange's data center. The latency is literally the time it takes for light to travel from the colocated server to the exchange server [10].

Exchange
Exchange Trading Engine
Client Gateway
Client Gateway
Client Gateway

Orders and executions are the most important entities in the exchange. We assume there's an API for these mainframed in the high-level design, or logical software view.

- In the critical trading path, orders and executions are not stored in a database. To achieve high performance, the path consumes messages from a memory-mapped log (shared memory) and writes to another memory-mapped log. Orders and executions are stored in the sequencer for fast recovery and are forwarded after the critical path. Another process, the sequencer, persists the orders and executions to a database.
- The market data publication process is also outside the critical path. It consumes messages from the sequencer and publishes them.
- Executions are forwarded to the market data processor to reconstruct the order book and candlestick chart data. We discuss these data types later.

Order book

An order book is a list of buy and sell orders for a specific security or financial instrument, organized by price level [72] [73]. It is a key data structure in the matching engine for fast order matching. An efficient data structure for an order book must satisfy these requirements:

- Constant time lookup. Retrieving orders should be independent of the number of price levels on the order book.
- Fast add/cancel/modifies operations, preferably O(1) time complexity. Operations include placing a new order, cancelling an order, and modifying an existing order.
- Fast update. Operations change an order.
- Query best bid/ask.
- Iterable through price levels.

Let's walk through an example order execution against an order book, as illustrated in Figure 73.

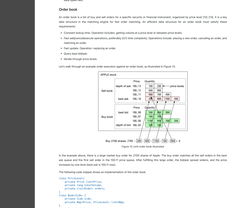


Figure 73 Limit order book of Binance futures

In the example above, there is a large market buy order for 270 units. The buy order matches all the sell orders in the best bid and ask price levels. The best ask price is now 150.01 (since, ignoring the stage order, the stopped sell orders, and the price increases by one tick (tick size) is 100.01 ticks).

The following code shows an implementation of the order book.

```
class Price(enum):
    PRICE_UNDEFINED = 0
    PRICE_DECIMALS = 2
    PRICE_TICK_SIZE = 1
class Side(enum):
    UNDEFINED = 0
    BUY = 1
    SELL = 2
class OrderStatus:
    UNDEFINED = 0
    PENDING = 1
    PARTIALLY_FILLED = 2
    FILLED = 3
    CANCELLED = 4
class OrderRecord:
    price: Price
    side: Side
    qty: int
    status: OrderStatus
    next_order_id: int
    prev_order_id: int
class PriceLevel:
    price: Price
    total_qty: int
    first_order_id: int
class OrderBook:
    price_levels: dict[Price, PriceLevel]
    orders: dict[int, OrderRecord]
    buy_tree: SortedDictionary[Price, int]
    sell_tree: SortedDictionary[Price, int]
    best_buy_price: Price
    best_sell_price: Price
    mark_price: Price
    mid_price: Price
    spread: Price
    buy_stop_orders: SortedDictionary[Price, int]
    sell_stop_orders: SortedDictionary[Price, int]
    buy_trailing_stop_orders: SortedDictionary[Price, int]
    sell_trailing_stop_orders: SortedDictionary[Price, int]
```

Since the code need all the design requirements listed above (lookup, iterable, order adding/cancelling a limit order, is the time complexity O(1)? The answer is no as we are using a sorted tree ("SortedDictionary" in the code). To have a more efficient order book, change the data structures of "orders" to a doubly-linked list so that the deletion type of operation (cancel and market) is also O(1), with random access to any element in the doubly-linked list still being O(n).

1. Placing a new limit order requires inserting a new node in the tree and updating the price level if it already exists.
2. Matching an order means deleting an Order from the header of the Price level. This is O(1) time complexity for a doubly-linked list.
3. Cancelling an order requires deleting an Order from the middle of the price level. This is O(1) time complexity for a doubly-linked list.
4. Updating an order means updating the Order in place if the new quantity is less than or equal to the old quantity. Otherwise, it is cancelled and then the Order is added to the tail. This is O(1) time complexity.
5. Yes, the code should have to traverse the entire tree to locate the previous price level in order to delete the order. That should have taken O(log(n)) time. Instead, we store the price level in the order object so that we can jump to the previous order, which allows the code to delete the price level in O(1) time. This is a classic trade-off between space and speed.

Figure 74 explains how these three operations work.

Figure 24 Market Data Publisher

This design utilizes ring buffers. A ring buffer, also called a circular buffer, is a fixed-size queue with the head connected to the tail. A producer continuously produces data and one or more consumers pull data off it. The space in a ring buffer is pre-allocated. There is no object creation or deallocation necessary. The data structure is also lock-free. There are other techniques to make the data structure even more efficient. For example, padding ensures that the ring buffer's sequence number is never in a cache line with anything else. Refer to [28] for more detail.

Distribution fairness of market data

In stock trading, having lower latency than others is like having an oracle that can see the future. For a regulated exchange, it is important to guarantee that all the receivers of market data get that data at the same time. Why is this important? For example, the MDP holds a list of data subscribers, and the order of the subscribers is decided by the order in which they connect to the publisher, with the first one always receiving data first. Guess what happens, then? Smart clients will fight to be the first on the list when the market opens.

There are some ways to mitigate this. Multicast using reliable UDP is a good solution to broadcast updates to many participants at once. The MDP could also assign a random order when the subscriber connects to it. We look at multicast in more detail.

Multicast

Data can be transported over the internet by three different types of protocols. Let's take a quick look.

1. Unicast: from one source to one destination.
2. Broadcast: from one source to an entire subnetwork.
3. Multicast: from one source to a set of hosts that can be on different subnetworks.

Multicast is a commonly-used protocol in exchange design. By configuring several receivers in the same multicast group, they will in theory receive data at the same time. However, UDP is an unreliable protocol and the datagram might not reach all the receivers. There are solutions to handle retransmission [29].

Colocation

While we are on the subject of fairness, it is a fact that a lot of exchanges offer colocation services, which put hedge funds or brokers' servers in the same data center as the exchange. The latency in placing an order to the matching engine is essentially proportional to the length of the cable. Colocation does not break the notion of fairness. It can be considered as a paid-for VIP service.

Network security

An exchange usually provides some public interfaces and a DDoS attack is a real challenge. Here are a few techniques to combat DDoS:

1. Isolate public services and data from private services, so DDoS attacks don't impact the most important clients. In case the same data is served, we can have multiple read-only copies to isolate problems.
2. Use a caching layer to store data that is infrequently updated. With good caching, most queries won't hit databases.
3. Harden URLs against DDoS attacks. For example, with an URL like `https://my.website.com/data?from=123&to=456`, an attacker can easily generate many different requests by changing the query string. Instead, URLs like this work better: `https://my.website.com/data/recent`. It can also be cached at the CDN layer.
4. An effective safelist/blocklist mechanism is needed. Many network gateway products provide this type of functionality.
5. Rate limiting is frequently used to defend against DDoS attacks.

Wrap Up

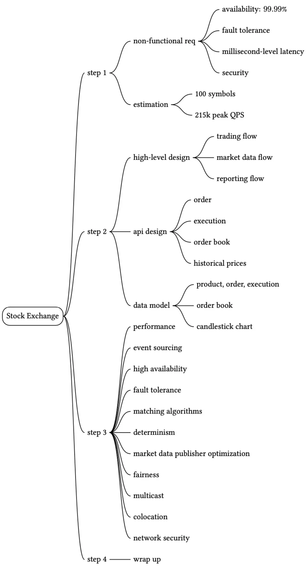
After reading this chapter, you may come to the conclusion that an ideal deployment model for a big exchange is to put everything on a single gigantic server or even one single process. Indeed, this is exactly how some exchanges are designed!

With the recent development of the cryptocurrency industry, many crypto exchanges use cloud infrastructure to deploy their services [30]. Some decentralized finance projects are based on the notion of AMM (Automatic Market Making) and don't even need an order book.

The convenience provided by the cloud ecosystem changes some of the designs and lowers the threshold for entering the industry. This will surely inject innovative energy into the financial world.

Congratulations on getting this far! Now give yourself a pat on the back. Good job!

Chapter Summary



Reference materials

[1] LMAX exchange was famous for its open-source Disruptor: https://www.lmax.com/exchange

[2] IEX attracts investors by "playing fair", also is the "Flash Boys Exchange": https://en.wikipedia.org/wiki/IEX

[3] NYSE matched volume: https://www.nyse.com/markets/us-equity-volumes

[4] HKEX daily trading volume: https://www.hkex.com.hk/Market-Data/Statistics/Consolidated-Reports/Securities-Statistics-Archive/Trading_Value_Volume_And_Number_Of_Deals?sc_lang=en#select1=0

[5] All of the World's Stock Exchanges by Size: http://money.visualcapitalist.com/all-of-the-worlds-stock-exchanges-by-size/

[6] Denial of service attack: https://en.wikipedia.org/wiki/Denial-of-service_attack

[7] Market impact: https://en.wikipedia.org/wiki/Market_impact

[8] Fix trading: https://www.fixtrading.org/

[9] Event Sourcing: https://martinfowler.com/eaaDev/EventSourcing.html

[10] CME Co-Location and Data Center Services: https://www.cmegroup.com/trading/colocation/co-location-services.html

[11] Epoch: https://www.epoch101.com/

[12] Order book: https://www.investopedia.com/terms/o/order-book.asp

[13] Order book: https://en.wikipedia.org/wiki/Order_book

[14] How to Build a Fast Limit Order Book: https://bit.ly/3nqMtEQ

[15] Developing with kdb+ and the q language: https://code.kx.com/q/

[16] Latency Numbers Every Programmer Should Know: https://gist.github.com/jboner/2841832

[17] mmap: https://en.wikipedia.org/wiki/Memory_map

[18] Context switch: https://bit.ly/3qva7A6

[19] Reliable User Datagram Protocol: https://en.wikipedia.org/wiki/Reliable_User_Datagram_Protocol

[20] Aeron: https://github.com/real-logic/aeron/wiki/Design-Overview

[21] Chaos engineering: https://en.wikipedia.org/wiki/Chaos_engineering

[22] Raft: https://raft.github.io/

[23] Designing for Understandability: the Raft Consensus Algorithm: https://raft.github.io/slides/uiuc2016.pdf

[24] Supported Matching Algorithms: https://bit.ly/3aYoCEo

[25] Dark pool: https://www.investopedia.com/terms/d/dark-pool.asp

[26] HdrHistogram: A High Dynamic Range Histogram: http://hdrhistogram.org/

[27] HotSpot (virtual machine): https://en.wikipedia.org/wiki/HotSpot_(virtual_machine)

[28] Cache line padding: https://bit.ly/3IZTFWz

[29] NACK-Oriented Reliable Multicast: https://en.wikipedia.org/wiki/NACK-Oriented_Reliable_Multicast

[30] AWS Coinbase Case Study: https://aws.amazon.com/solutions/case-studies/coinbase/

Mark as Complete

Partner With Us
Be an Affiliate
Become a Contributor

Company & Legal
Our Team
Newsletter
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information

Support
hi@bytebytego.com
Report a Bug

Resources
YouTube
Visual Dev Guides
Find Jobs
Prepare for Coding Interviews

Ask Alex

Copyright ©2022-2026 ByteByteGo Inc. All rights reserved.